



CONNECT DESK

21CY711-PROJECT PHASE-I



Submitted by

ARAVINTH S	(714022107009)
DHARUN J	(714022107018)
HAREE CHAND VELAVAN S	(714022107022)
KISHOR M V S	(714022107031)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)

SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution, Accredited by NAAC with “A” Grade)

COIMBATORE - 641 062

ANNA UNIVERSITY: CHENNAI 600 025

NOV- 2025

BONAFIDE CERTIFICATE

ANNA UNIVERSITY: CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “**CONNECT DESK** ” is the bonafide work of **ARAVINTH S (714022107009)**, **DHARUN J (714022107018)**, **HAREE CHAND VELAVAN S (714022107022)**, **KISHOR M V S (714022107031)** who carried out the project under my supervision.

SIGNATURE

MS. SABITHA K

ASSISTANT PROFESSOR

SUPERVISOR

Department of Computer Science
and Engineering

(Cyber Security)

Sri Shakthi Institute Of

Engineering and Technology

Coimbatore – 641 062

SIGNATURE

DR.S. SUBASREE

HEAD OF THE DEPARTMENT

Department of Computer Science
and Engineering

(Cyber Security)

Sri Shakthi Institute Of

Engineering and Technology

Coimbatore – 641 062

Submitted for the University Mini Project Viva-Voce conducted on _____

EXAMINER

ACKNOWLEDGEMENT

ACKNOWLEDGEMENT

We express our deepest gratitude to our Chairman **Dr.S. Thangavelu**, for his continuous encouragement and support throughout the course of study . We are thankful to our Secretary **Er.T. Dheepan** and Joint Secretary **Mr.T. Sheelan** for their encouragement.

We would like to express our gratitude to our Principal **Dr.N.K. Sakthivel** for his academic interest shown towards the students, We are very grateful to our **Dr.S. Subasree**, Dean Academics and Head of the Department of Computer Science and Engineering (Cyber Security) for providing us with the necessary facilities.

It's a great pleasure to thank our Project Guide **Ms.Sabitha K** Assistant Professor, Department of Computer Science and Engineering (Cyber Security) for his valuable technical suggestion and guidance throughout this project work.

We solemnly extend our thanks to all the teaching and non-teaching staff of our department, family and friends for their valuable support.

ARAVINTH S	(714022107009)
DHARUN J	(714022107018)
HAREE CHAND VELAVAN S	(714022107022)
KISHOR M V S	(714022107031)

ABSTRACT

ABSTRACT

Connect Desk is an innovative, real-time communication platform that contains the essential features of Slack while achieving the functions of modern web technologies. Users can create workspaces, manage channels, send messages, and collaborate using text and video tools. It is constructed with the MERN stack, which is composed of MongoDB, Express.js, React, and Node.js, providing scalability, flexibility, and performance. Real-time messaging and video calling features are accomplished with Socket.IO and Stream to provide high-quality and uninterrupted communication. Authentication and user access flows through Clerk to assure secure handling of customers. Other technologies such as Tailwind CSS, Zustand, Sentry, and RabbitMQ perform handling state, designing the interface, monitoring, and managing background processing. Connect Desk was designed and developed with modularity, to have a customizable and easily integrated in various organizational contexts. Connect Desk demonstrates the development of a fully functional, open-source communication system that imitates Slack, giving secure, real-time collaborative capabilities for teams and institutions. Connect Desk demonstrates modern JavaScript technologies for developing scalable and interactive applications that service the modern workspace.

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	I
	LIST OF ABBREVIATIONS	II
	LIST OF FIGURES	III
1	INTRODUCTION	1
1.1	PROBLEM STATEMENT	2
1.2	PROJECT OBJECTIVES	2
1.3	SCOPE OF THE PROJECT	3
2	LITERATURE REVIEW	4
2.1	EXISTING SYSTEM ANALYSIS	4
2.2	PROPOSED SYSTEM	5
2.3	TECHNOLOGY STACK RATIONALE	5
2.3.1	MERN STACK	5
2.3.2	OTHER KEY TECHNOLOGIES	6
3	SYSTEM REQUIREMENTS	8
3.1	FUNCTIONAL REQUIREMENTS	8
3.2	NON-FUNCTIONAL REQUIREMENTS	9
3.3	SOFTWARE REQUIREMENTS	10
3.4	HARDWARE REQUIREMENTS	10
4	SYSTEM DESIGN AND ARCHITECTURE	11
4.1	SYSTEM ARCHITECTURE	11
4.2	DATABASE DESIGN	12
4.3	AUTHENTICATION FLOW	13
4.4	REAL TIME COMMUNICATION DESIGN	16

4.5	UML DIAGRAM	17
5	IMPLEMENTATION	19
5.1	MODULE 1: SERVER AND API SETUP	19
5.2	MODULE 2: USER AUTHENTICATION	19
5.3	MODULE 3: CLIENT-SIDE UI	20
5.4	MODULE 4: STATE MANAGEMENT	20
5.5	MODULE 5: REAL-TIME MESSAGING	21
5.6	MODULE 6: FILE UPLOADS	21
6	TESTING AND RESULTS	23
6.1	TESTING STRATEGY	23
6.2	RESULTS	24
7	CONCLUSION AND FUTURE SCOPE	27
7.1	CONCLUSION	27
7.2	FUTURE ENHANCEMENT	27
	REFERENCES	29

LIST OF ABBREVIATIONS

LIST OF ABBREVIATIONS

MI	MongoDB, Express.js, React, Node.js
AP	Application Programming Interface
RE	Representational State Transfer
HT	HyperText Transfer Protocol
JW	JSON Web Token
SP	Single Page Application
UI	User Interface
UX	User Experience
CR	Create, Read, Update, Delete
DB	Database
OE	Object Data Modeling
CD	Content Delivery Network
XS	Cross-Site Scripting
CS	Cross-Site Request Forgery
RA	Random Access Memory
CP	Central Processing Unit
OS	Operating System

LIST OF FIGURES

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
4.1	Connect Desk System Architecture	12
4.2	MongoDB E-R Diagram	13
4.3	JWT Authentication Flow	15
4.4	Socket.io Real-time Data Flow	16
4.5	Use Case Diagram	17
4.6	Sequence Diagram for Sending a Message	18
6.1	Login and Registration Page	24
6.2	Main Application Chat Interface	25
6.3	File Upload Demonstration	26

CHAPTER 1

INTRODUCTION

In today's corporate and development environments, real-time communication is essential for boosting productivity, supporting agile workflows, and building team cohesion. It involves quickly sharing messages, files, and project-related information within an organization. Traditionally, proprietary, subscription-based platforms like Slack and Microsoft Teams have dominated this space. While these services are effective, they can be too costly for startups and small teams. They also require companies to keep their sensitive internal discussions and proprietary files on third-party servers, which raises serious concerns about data privacy and sovereignty.

To address this issue, the "Connect Desk" project was created as a self-hosted, open-source, and full-stack solution focused on data control and cost efficiency. This application is designed as a high-fidelity clone of Slack and is built entirely on the MERN (MongoDB, Express.js, React, Node.js) stack. It offers a strong and scalable platform that organizations can run on their own private infrastructure. The application features a user-friendly, web-based interface that replicates the functions of its commercial counterparts. Instead of relying on a third-party vendor and paying monthly fees, users can handle all their internal communications in a secure environment controlled by the company.

"Connect Desk" supports and simplifies various important collaboration tasks. These include secure user registration and authentication through Passport.js and JWT, channel-based messaging (both public and private), one-on-one direct messaging, and real-time message broadcasting with Socket.io. It also incorporates Cloudinary for smooth media file uploads, while the application's complex state is managed.

These features enable teams—especially startups, open-source projects, and organizations that prioritize privacy—to create and manage their own collaboration hub without a hefty financial burden. By offering a comprehensive, enterprise-grade chat solution built on the well-known and powerful MERN stack, "Connect Desk" makes it easier for teams to use self-hosted communication tools. It is an excellent platform for organizations that want to maintain full control over their data while encouraging efficient, immediate, and smooth teamwork.

1.1 PROBLEM STATEMENT:

In today's digital and often remote work environment, effective communication is essential. Traditional tools like email are slow and lead to scattered conversations and messy inboxes. While commercial options like Slack, Microsoft Teams, and Discord have addressed these issues, they also come with their own problems:

- Cost:** For large teams or premium features, these services can be pricey. This can be a hurdle for startups, open-source projects, and small businesses.
- Data Control:** Data is kept on third-party servers, raising concerns for organizations that have strict data privacy and compliance needs.
- Vendor Lock-in:** Depending on a specific platform makes it tough to switch or connect with other systems.

The key issue is the need for an open-source, self-hosted, and affordable collaboration platform that offers the main features of a modern chat application without the costs and data control concerns.

1.2 PROJECT OBJECTIVES:

The main goal of this project is to create a fully functional Slack clone that solves the identified problem. The specific objectives are as follows:

- To design a scalable and efficient client-server setup for a real-time messaging app using the MERN stack.
- To implement a secure user authentication system using standard practices, particularly Passport.js and JSON Web Tokens (JWT).
- To develop a chat interface with features using React, allowing users to create channels, join channels, and send or receive messages.
- To integrate a real-time communication layer with Socket.io to ensure instant message delivery.
- To implement a strong client-side state management system using Redux and Redux-Saga to manage application state and handle asynchronous tasks.
- To integrate a third-party cloud service (Cloudinary) for managing image and file uploads, reducing storage needs on the application server.

1.3 SCOPE OF THE PROJECT:

The scope of "Connect Desk" includes the key features of a modern chat application.

Features included:

- User registration and login.
- Secure, token-based authentication (JWT).
- Creation of public and private channels.
- Ability to browse and join existing public channels.
- Real-time text messaging within channels.
- Direct messaging (one-on-one) between users.
- Uploading and displaying image/media files.
- Displaying a list of users in the current channel.

Features excluded from this implementation:

- Video and voice calls, which would require WebRTC.
- Threaded message replies.
- User notifications (@mentions).
- Search functionality across all messages.
- Integration with third-party applications, such as Giphy and Google Drive.
- Advanced administrative roles and permissions.

CHAPTER 2

LITERATURE REVIEW

A careful look at current solutions and the choice of the right technology stack are necessary for successful project development. This chapter starts by reviewing the main existing system, Slack. This helps to set a baseline and pinpoint its main features, strengths, and weaknesses, especially regarding cost and data privacy. After this analysis, we introduce the proposed "Connect Desk" system as a direct answer to these shortcomings. The rest of the chapter focuses on explaining the chosen technology stack. It provides reasons for picking the MERN stack for full-stack development, Socket.io for real-time features, Redux and Redux-Saga for managing state, Passport.js and JWT for security, and Cloudinary for handling media. It shows how each part supports the project's goals.

2.1 EXISTING SYSTEM ANALYSIS:

Slack is a leading tool for team collaboration that brings all communication within an organization into one place. It uses a "workspace" model, which is further divided into "channels" for topic-based conversations and "direct messages" for private communication.

Strengths:

- **User Interface:** Slack is known for its polished, user-friendly interface, which is easy to learn.
- **Integrations:** Its main strength lies in a large ecosystem of third-party app integrations, such as GitHub, Google Drive, and Jira. This allows it to act as a central hub for all team activities.
- **Features:** It offers many features, including threads, video and audio calls, huddles, and a powerful universal search.

Weaknesses:

- **Cost:** The free tier is very limited, especially when it comes to message history. Paid plans can be costly and increase with the number of users, leading to significant operational expenses.
- **Data Control:** As a closed-source SaaS product, all data, messages, and files are stored on Slack's servers. This may not meet the needs of organizations that have high security or data sovereignty requirements.

2.2 PROPOSED SYSTEM:

"Connect Desk" is introduced as an open-source, self-hosted alternative that offers the basic features of Slack. It includes channels, direct messages, and real-time chat in a package that organizations can set up on their own systems. The benefits of this proposed system are:

- **Full Data Control:** All data, including users and messages, is stored in the organization's own MongoDB instance.
- **No Cost:** Since it is open-source, there are no licensing fees or charges based on the number of users, just a consistent hosting cost.
- **Customizability:** The open-source MERN stack makes it easy to modify and add features.

2.3 TECHNOLOGY STACK RATIONALE:

The team chose the technology stack to build a modern, high-performance application that can scale. They will use one language, JavaScript, throughout the entire stack.

2.3.1 MERN Stack:

The MERN stack includes a set of JavaScript-based technologies for building full-stack web applications. It consists of MongoDB, Express.js, React, and Node.js.

- **Node.js (Backend Runtime):** Node.js is a server-side runtime environment that runs JavaScript code outside of a web browser. It is built on Chrome's V8 JavaScript engine and uses an event-driven, non-blocking I/O model. This model works well for a chat application, which requires heavy input/output operations and must manage thousands of ongoing connections without delays. Its extensive npm (Node Package Manager) ecosystem offers pre-built modules for tasks like authentication and database connections.
- **Express.js (Backend Framework):** Express.js is a straightforward and flexible web application framework for Node.js. It adds an easy layer on top of Node.js for handling HTTP requests and responses. Express is used to create the entire RESTful API for "Connect Desk." It allows developers to define routes (e.g., /api/users/login), integrate middleware (such as Passport for authentication or CORS for cross-origin requests), and serve static files. Its simplicity and

effectiveness make it the go-to choice for building APIs in the Node.js environment.

- **React (Frontend Library):** React is a declarative JavaScript library for creating user interfaces, especially for Single Page Applications (SPAs). Developed by Facebook, its main strength lies in its component-based architecture. The UI of "Connect Desk" is divided into reusable components like Sidebar, ChatWindow, and MessageInput. React uses a "virtual DOM" to update the UI efficiently; when a new message comes in, React determines the least number of changes needed and updates only that part of the real DOM, resulting in a fast and responsive user experience.
- **MongoDB (Database):** MongoDB is a general-purpose NoSQL document database. Unlike traditional SQL databases that use tables and rows, MongoDB keeps data in flexible, JSON-like documents. This schema-less model is perfect for a chat application where data structures may change. A "message" document can store text, a timestamp, and a user ID, and later be updated to include an optional file URL or an array of reactions without needing complicated database migrations. Mongoose, an Object Data Modeling (ODM) library, provides a schema and validation layer on top of MongoDB.

2.3.2 Other Key Technologies:

- **Socket.io (Real-time Engine):** Socket.io is a library that enables real-time, two-way event-based communication. It powers the "instant" messaging feature in "Connect Desk." While a traditional HTTP API is good for fetching data (like logging in), it uses a request-response protocol. Socket.io, on the other hand, sets up a continuous, low-latency connection between the client and server. When a user sends a message, the client triggers a sendMessage event. The server listens for this event, saves the message, and then broadcasts a newMessage event to all other clients in that specific channel, instantly delivering the data.
- **Redux & Redux-Saga (State Management):**
 - **Redux:** Redux is a state container for JavaScript applications, typically used with React. In complex applications like a chat client, many components need to share state (e.g., who is logged in, which channel is active, what messages belong to that channel). Redux provides a centralized "store" for this global state.
 - **Redux-Saga:** Redux-Saga is middleware for Redux that deals with asynchronous actions, or side effects. When a user wants to log in, a LOGIN_REQUEST action is sent. A Saga "watcher" intercepts this action, makes the asynchronous API call to the server, and

then dispatches either a LOGIN_SUCCESS or LOGIN_FAILURE action based on the response. This approach keeps the React components clean and free of asynchronous logic.

- **Passport.js & JWT (Authentication):**

- **Passport.js:** IT is authentication middleware for Node.js. It is modular and adaptable. For "Connect Desk," it is used to set up strategies for authentication. A passport-local strategy checks a user's email and password against the database during login.

- **JSON Web Token (JWT):** JWT is a compact, URL-safe standard for securely sharing information as a JSON object. After a user logs in successfully, the server generates and signs a JWT (including the user's ID) and sends it to the client. The client stores this token and includes it in the Authorization header of every future API request. The server then uses a passport-jwt strategy to verify this token and identify the user, securing the API.

- **Cloudinary (Media Management):** Cloudinary is a cloud service for managing images and videos. Storing user-uploaded files directly on the server's filesystem or in the database is inefficient and doesn't scale well. In this project, when a user uploads an image, the backend server sends the file to the Cloudinary API. Cloudinary stores the file, optimizes it, and provides a CDN-hosted URL. The server then saves only this URL string to the Message document in MongoDB.

CHAPTER 3

SYSTEM REQUIREMENT

This chapter explains the specifications and limits of the "Connect Desk" application. It acts as an agreement between the developers and stakeholders. It turns the project goals into a set of clear criteria. The chapter is divided into functional requirements, which describe the specific behaviors and features of the system, like user authentication and messaging. It also includes non-functional requirements, which define the system's quality attributes, such as performance, security, and usability. Lastly, it lists the specific software and hardware needed for the development, deployment, and operation of the application, providing a clear understanding of the technical environment.

3.1 FUNCTIONAL REQUIREMENTS:

Functional requirements define the specific behaviors and functions of the system.

User Authentication:

- The system will have a registration page where new users can create an account with a username, email, and password.
- It will also provide a login page for existing users.
- The system will check all registration and login fields for correctness.
- User passwords will be securely hashed and salted before being stored in the database.
- The system will issue a JWT after a successful login.
- It will protect all important routes and API endpoints, requiring a valid JWT for access.
- Users will have the option to log out.

Channel Management:

- The system will let logged-in users create a new channel, specifying a name and whether it is public or private.
- It will show a list of all public channels to logged-in users.
- Users can join any public channel.
- Private channels will only be visible to users who are members of that channel.

Real-time Messaging:

- The system will let users pick a channel from their list to view its messages.
- It will load and display the message history for the selected channel.
- A text input box will be provided for users to compose messages.
- When a user sends a message, it will be instantly broadcast to all other active users in that channel.
- The system will store all sent messages in the database.

Direct Messaging (DM):

- The system will allow a user to select another user from a list for one-on-one direct messaging.
- All messages sent in a DM will be private and visible only to the two users involved.

File Upload:

- The system will provide a button or a drag-and-drop feature for uploading image files.
- Uploaded images will be displayed inline within the chat window.

3.2 NON-FUNCTIONAL REQUIREMENTS:

Non-functional requirements define the quality attributes and constraints of the system.

● **Performance:** Real-time messages must have a perceived latency of less than 1 second. API endpoints for fetching message history must respond within 2 seconds.

● **Security:** User passwords must be hashed. Communication between the client and server must occur over HTTPS in production. All API routes must be secure.

● **Usability:** The UI must be clean, intuitive, and responsive, and it must work on all major modern web browsers, including Chrome, Firefox, Safari, and Edge.

● **Scalability:** The application architecture, which includes a stateless API and a separate database, must be able to scale horizontally to support a growing number of concurrent users.

● **Reliability:** The application must handle server or network disconnections smoothly and attempt to reconnect the WebSocket automatically.

3.3 HARDWARE REQUIREMENTS:

Development:

- **CPU:** Dual-core 2.0 GHz
- **RAM:** 8 GB
- **Storage:** 50 GB SSD
- **Network:** Internet access

Production Server:

- **CPU:** 2 vCPU
- **RAM:** 4 GB
- **Storage:** 50 GB SSD
- **Network:** High-speed internet

3.4 SOFTWARE REQUIREMENTS:

- **Operating System:** Windows 10+, macOS 11+, Linux
- **Backend:** Node.js v18+, Express.js v4+, Mongoose v7+, Socket.io v4+, Passport.js v0.6+, JWT & Bcrypt.js, Cloudinary v2+
- **Frontend:** React v18+, Redux v4+ & React-Redux v8+, Redux-Saga v1+, React Router v6+, Axios v1+, Socket.io-client v4+, Styled-Components v5+
- **Database:** MongoDB v6+
- **Development:** Git, VS Code (or other IDE), Postman, Web Browser

CHAPTER 4

METHODOLOGY

This chapter outlines the technical plan for the "Connect Desk" application. It shifts from the requirements to the solution. It explains the system architecture at a high level. The focus is on the client-server model and how the MERN stack components interact. The chapter also covers the database design and provides the MongoDB schema for important collections, such as users, channels, and messages. It details the key data flows, including the user authentication process with JWT and the real-time communication flow handled by Socket.io. This design is made clearer with standard UML diagrams, which show user interactions and system processes visually.

4.1 SYSTEM ARCHITECTURE OVERVIEW:

The application uses a decoupled client-server design.

1. Client (Frontend): This is a React-based Single Page Application (SPA) that runs entirely in the user's browser. It handles rendering the UI and managing user interactions. It communicates with the backend through two separate channels:

1.1. RESTful API (HTTP): This channel is for stateless, request-response actions such as logging in, registering, fetching channel lists, and loading message history.

1.2. WebSockets (Socket.io): This channel is for stateful, real-time, event-based actions, mainly for sending and receiving new messages.

2. Server (Backend): This is a Node.js application that provides both the RESTful API (using Express.js) and the WebSocket server (using Socket.io). It takes care of all business logic, authentication, and database interactions.

3. Database (Persistence): This is a MongoDB instance that stores all application data. The backend server accesses it exclusively.

4. External Services: This includes third-party services like Cloudinary, which the backend calls to manage specific tasks such as file storage.

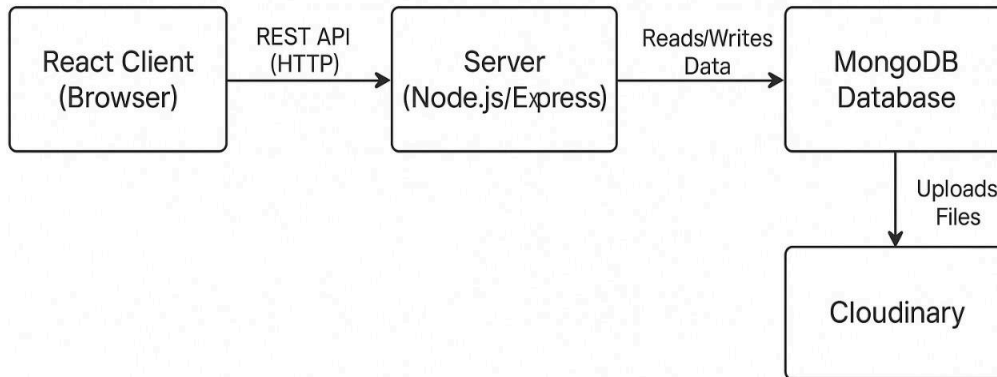


Fig 4.1 Connect Desk System Architecture

4.2 DATABASE DESIGN:

The database has three main collections, defined using Mongoose schemas.

● **User Collection:** This stores information for each registered user.

- username (String, required, unique): The user's display name.
- email (String, required, unique): The user's login email.
- password (String, required): The user's hashed password.
- createdAt (Date, default: Date.now): Timestamp for registration.

● **Channel Collection:** This stores information about each channel. It is also used for Direct Messages.

- name (String): The channel's display name (e.g., "#general").
- members (Array of ObjectId, ref: 'User'): A list of user IDs that are part of this channel.

- isDM (Boolean, default: false): A flag to indicate if this is a DM. If true, the name is usually null and members contain exactly two user IDs.
- createdAt (Date, default: Date.now): Timestamp for creation.

● **Message Collection:** This stores every individual message sent.

- channel (ObjectId, ref: 'Channel', required): The ID of the channel or DM this message belongs to.
- sender (ObjectId, ref: 'User', required): The ID of the user who sent the message.
- content (String): The text content of the message.
- fileUrl (String): An optional URL (from Cloudinary) if the message includes a file.
- createdAt (Date, default: Date.now): Timestamp for when the message was sent.

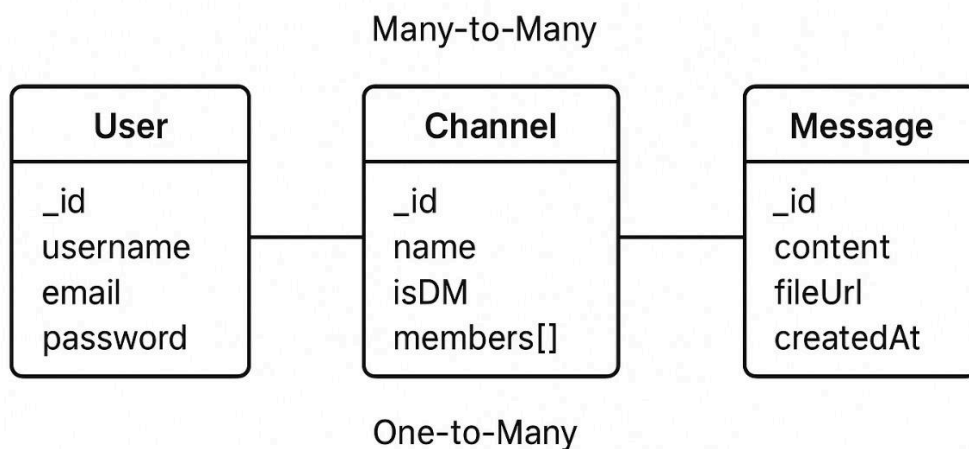


Fig 4.2 MongoDB E-R Diagram

4.3 AUTHENTICATION FLOW:

The system uses stateless JSON Web Token authentication.

- **Login Request:** The user enters their email and password from the React client.
- **API Call:** The client sends a POST request to the /api/auth/login endpoint on the Express server.

- **Server Validation:** The server locates the user in the MongoDB database using their email.
- **Password Check:** It uses `bcrypt.compare()` to determine if the submitted password matches the hashed password in the database.
- **Token Generation:** If the passwords match, the server creates a new JWT using `jwt.sign()`, placing the user's ID (`_id`) in the token's payload.
- **Token Response:** The server sends this token back to the client.
- **Client Storage:** The React client saves this token in `localStorage`.
- **Authenticated Requests:** For future API requests, such as creating a channel, the client retrieves the token from `localStorage` and includes it in the Authorization header. The server uses `passport-jwt` middleware to check the token, identify the user, and approve the request.

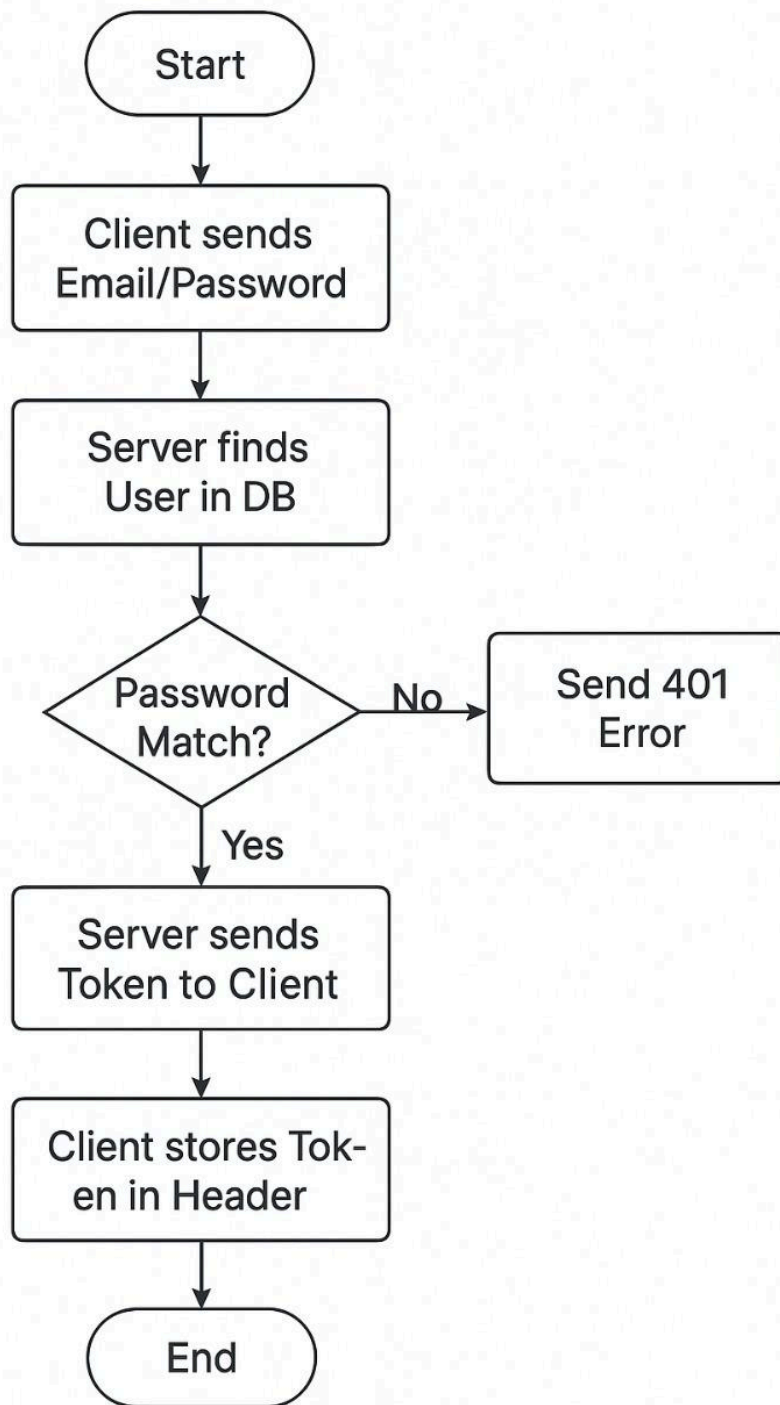


Fig 4.3 JWT Authentication Flow

4.4 REAL-TIME COMMUNICATION DESIGN:

This flow explains how User A sends a message and User B receives it in real time.

- **Connection:** When a user logs in and opens the chat app, the React client sets up a WebSocket connection to the Socket.io server.
- **Joining a Room:** When the user clicks on a channel (for example, "#general"), the client sends a `joinChannel` event with the channel's ID. The server receives this and uses `socket.join(channelId)` to add the user's socket to a room named after the channel ID.
- **Sending a Message:** User A types a message and hits send.
- **Client Emit:** User A's client sends a `sendMessage` event. This event includes the `channelId`, message content, and their user ID.
- **Server Receive:** The server's Socket.io listener for `sendMessage` activates.
- **Database Save:** The server creates a new `Message` object with the data and saves it to the MongoDB database.
- **Server Broadcast:** After saving, the server shares the new message with everyone in that room by using `io.to(channelId).emit('newMessage', savedMessageObject)`.
- **Receive a Message:** User B's client, which is also in the room for that `channelId`, receives the `newMessage` event. The event handler adds the new message to the Redux store, and React automatically updates the chat window to display it.

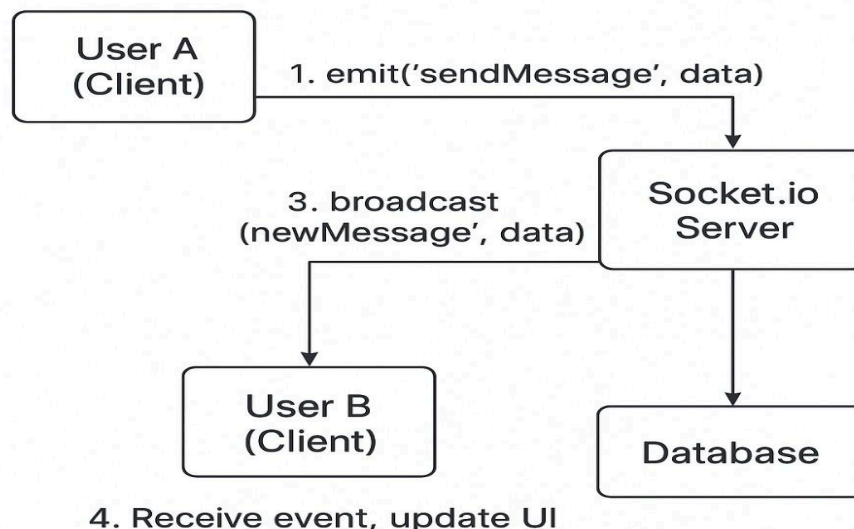


Fig 4.4 Socket.io & Real Time Dataflow

4.5 UML DIGRAMS:

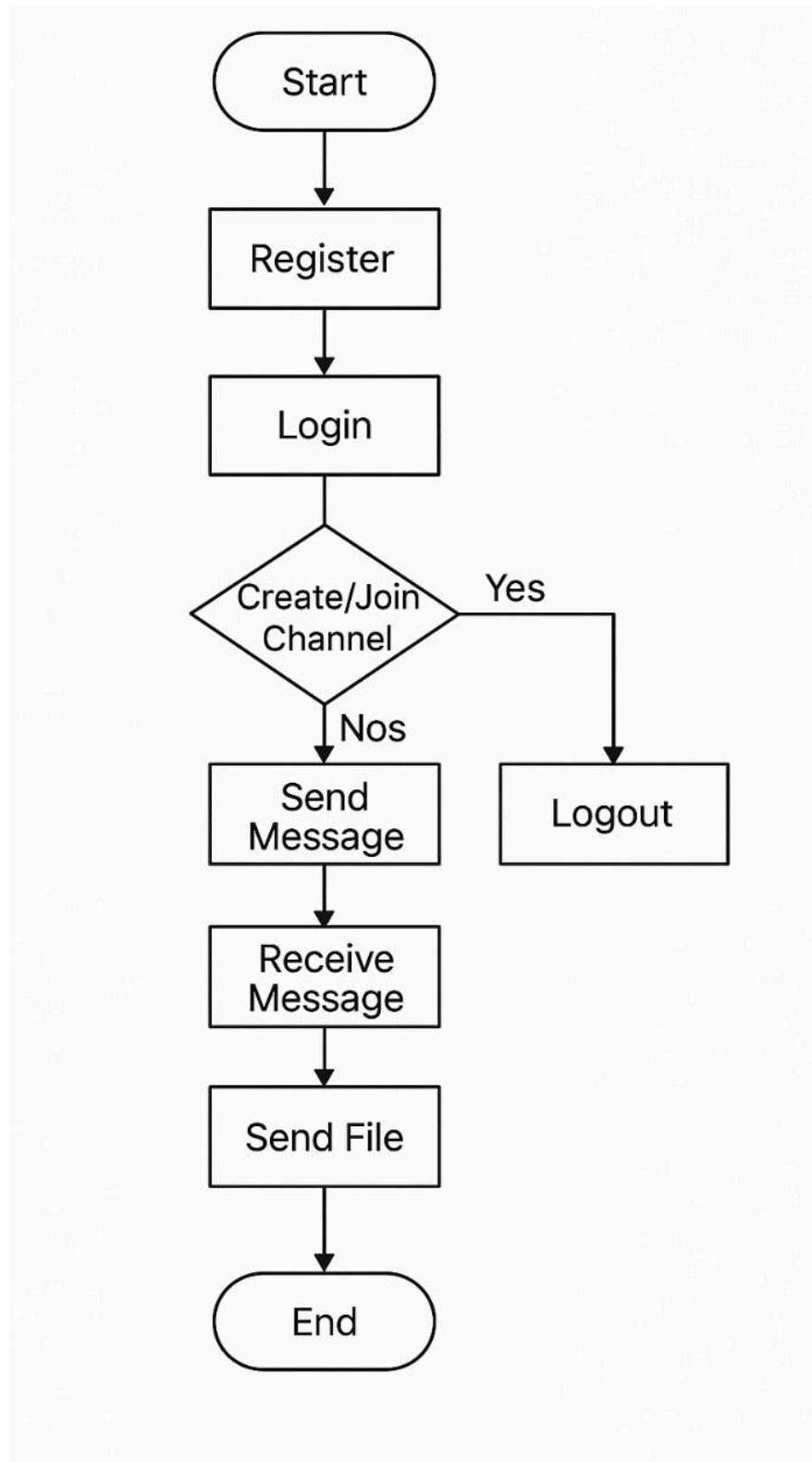


Fig 4.5 Use Case Diagram

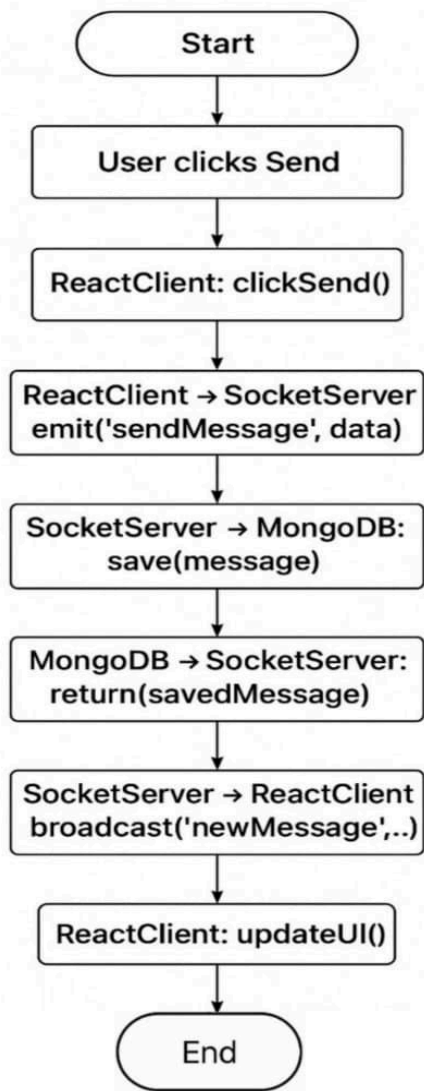


Fig 4.6 Sequence Diagram for Sending a Message

CHAPTER 5

IMPLEMENTATION

Following the design phase, this chapter outlines how to build the "Connect Desk" application. It turns the plans from Chapter 4 into working code. The implementation appears as a series of modules. Each module corresponds to a key part of the system. This includes setting up the backend server and RESTful API with Node.js and Express.js, creating the secure authentication module, and building the client-side interface with React and Styled Components. Important sections focus on integrating the Redux state management system and configuring the Socket.io client and server to enable real-time functionalities. This provides insight into the project's most crucial code segments.

5.1 MODULE 1: SERVER AND API SETUP:

The backend is initialized in `server.js`. This file:

1. Import `express`, `http`, `socket.io`, `mongoose`, `cors`, and `dotenv`.
2. Creates an Express app.
3. Wraps the app in a standard Node.js `http.createServer()` to be able to attach Socket.io.
4. Initializes a new socket.io server attached to the `httpServer`.
5. Applies middleware: `cors()` to allow cross-origin requests from the React client, and `express.json()` to parse request bodies.
6. Connects to the MongoDB database using `mongoose.connect(process.env.MONGO_URI)`.
7. Mounts the API routers: `app.use('/api/auth', authRoutes)`, `app.use('/api/channels', channelRoutes)`.
8. Starts the `httpServer` (not the app) to listen on a port, so that both Express and Socket.io run on the same port.

5.2 MODULE 2: USER AUTHENTICATION:

This module handles secure registration and login.

• **Registration (/api/auth/register):**

- (1) The route receives username, email, and password from `req.body`.
- (2) It checks if a user with that email already exists.

- (3) It generates a salt: `const salt = await bcrypt.genSalt(10);`
- (4) It hashes the password: `const hashedPassword = await bcrypt.hash(req.body.password, salt);`
- (5) A new User is created with the hashedPassword and saved to MongoDB.

●**Login (/api/auth/login):**

- (1) The route receives email and password.
- (2) It finds the user by email. If not found, it returns an error.
- (3) It compares the passwords: `const isMatch = await bcrypt.compare(req.body.password, user.password);`
- (4) If isMatch is false, it returns an error.
- (5) If isMatch is true, it creates a JWT payload: `const payload = { id: user.id };`
- (6) It signs the token: `jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '1d' }, (token,`
- (7) It sends the token back to the client.

5.3 MODULE 3: CLIENT-SIDE UI:

The frontend is built with create-react-app.

●**Component Structure:** The UI is broken into components:

- App.js: Contains the main ReactRouter logic.
- Login.js / Register.js: Forms for authentication.
- Home.js: A protected route that renders the main chat layout.
- Layout.js: A container holding the Sidebar and ChatWindow.
- Sidebar.js: Renders the list of channels and DMs.
- ChatWindow.js: Renders the MessageList and MessageInput for the active channel.
 - MessageList.js: Maps over the array of messages and renders a Message component for each.
 - MessageInput.js: The text-box for sending messages.

●**Styling (styled-components):** Instead of separate .css files, styles are written as React components. This co-locates styles with their components, making them reusable and preventing style conflicts.

5.4 MODULE 4: STATE MANAGEMENT:

●**ReduxStore:** A central store is created using `createStore()` and app Middleware(`sagaMiddleware`).

- **Reducers:** Pure functions that update state. For example, `messageReducer` handles actions like `FETCH_MESSAGES_SUCCESS` (to load history) and `RECEIVE_NEW_MESSAGE` (from `Socket.io`).
- **Sagas:** Generator functions handle async logic.
- **Watcher Saga:** `takeLatest('FETCH_MESSAGES_REQUEST', fetchMessagesSaga);`

5.5 MODULE 5: REAL-TIME MESSAGING:

- **Server-Side:**
 - **Save message to MongoDB:** `const newMessage = new Message({ ...data }); const savedMessage = await newMessage.save();`
 - **Populate sender details:** `await savedMessage.populate('sender', 'username');`
 - **Broadcast to the room:** `io.to(data.channelId).emit('newMessage', savedMessage);` }
- **Client-Side:**
 - **Connection:** A single socket instance is created: `const socket = io('http://localhost:5000');`
 - **Listening:** In a `useEffect` hook, the client listens for new messages.
 - **Sending:** The `MessageInput` component, on submit, emits the `sendMessage` event.

5.6 MODULE 6: FILE UPLOADS:

This process is split into two steps to avoid overloading the `WebSocket`.

1. Client-Side (Step 1 - Upload):

- When a user selects a file, the client creates `FormData`.
- It sends this `FormData` to a new REST API endpoint (e.g., `/api/upload/image`) using `axios.post()`.

2. Server-Side (REST Endpoint):

- This endpoint uses `multer` middleware to parse the file from the `FormData`.

- It uses the `cloudinary.uploader.upload_stream()` to pipe the file buffer directly to Cloudinary.
- Cloudinary responds with a result object containing `result.secure_url`.
- The server sends this `{ url: result.secure_url }` back to the client as the API response.

3. **Client-Side (Step 2 - Send Message):**

- The axios call now has the url.
- The client then calls `socket.emit('sendMessage', ...)` as normal, but this time the message object containing the `fileUrl` has empty text content.

4. **Server-Side (Socket):**

- The `sendMessage` socket handler receives this, saves the message with the `fileUrl` to MongoDB, and broadcasts it just like a text message.

CHAPTER 6

TESTING AND RESULTS

This chapter shows that the system works correctly and meets the requirements from Chapter 3. It starts by describing the testing strategy. This includes unit testing for individual parts, integration testing to make sure modules work together, and system-level testing to check user scenarios from start to finish. A set of detailed test cases is provided, showing the inputs, expected outputs, and actual results for key features like login, messaging, and file uploads. The chapter ends with the final testing results, supported by screenshots of the live application, which provide visual proof of the working product.

6.1 TESTING STRATEGY:

A multi-layered testing strategy was implemented:

- **Backend API Testing (Integration):** We used Postman a lot to manually test each API endpoint (/register, /login, /channels/create, /messages/:channelId, etc.). This ensured that the server-side logic, database interactions, and authentication middleware were all functioning correctly before we built the client.
- **Frontend Component Testing (Unit):** We wrote basic unit tests for individual React components using Jest and React Testing Library to confirm that they render properly with specific props.
- **End-to-End (E2E) Testing (Manual):** This was our main testing method. We ran the backend server and the frontend client at the same time to mimic real user scenarios. We used two different web browsers to simulate two users and test the real-time functionality.

6.2 RESULTS:

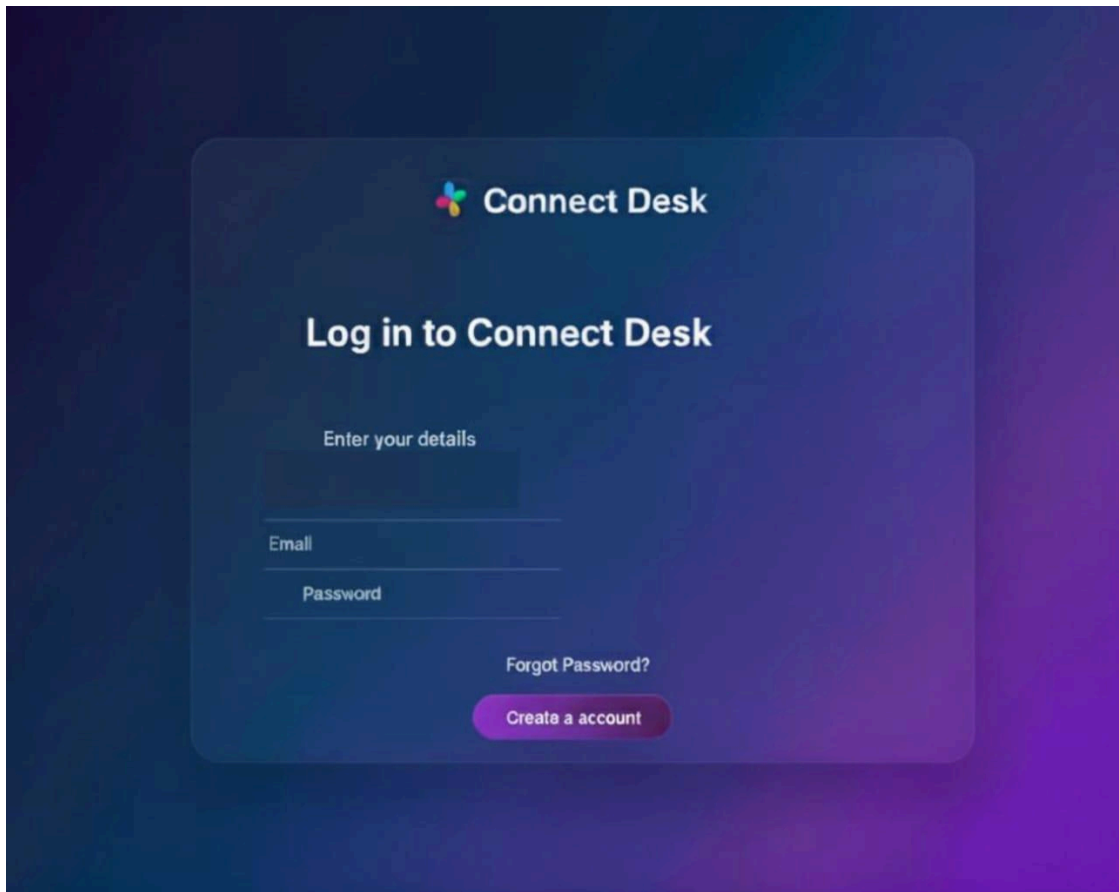


Figure 6.1: Login and Registration Page (Description) This screenshot shows the main authentication page. It features a clean, minimal form where users can log in with their email and password or switch to the registration form to create a new account.

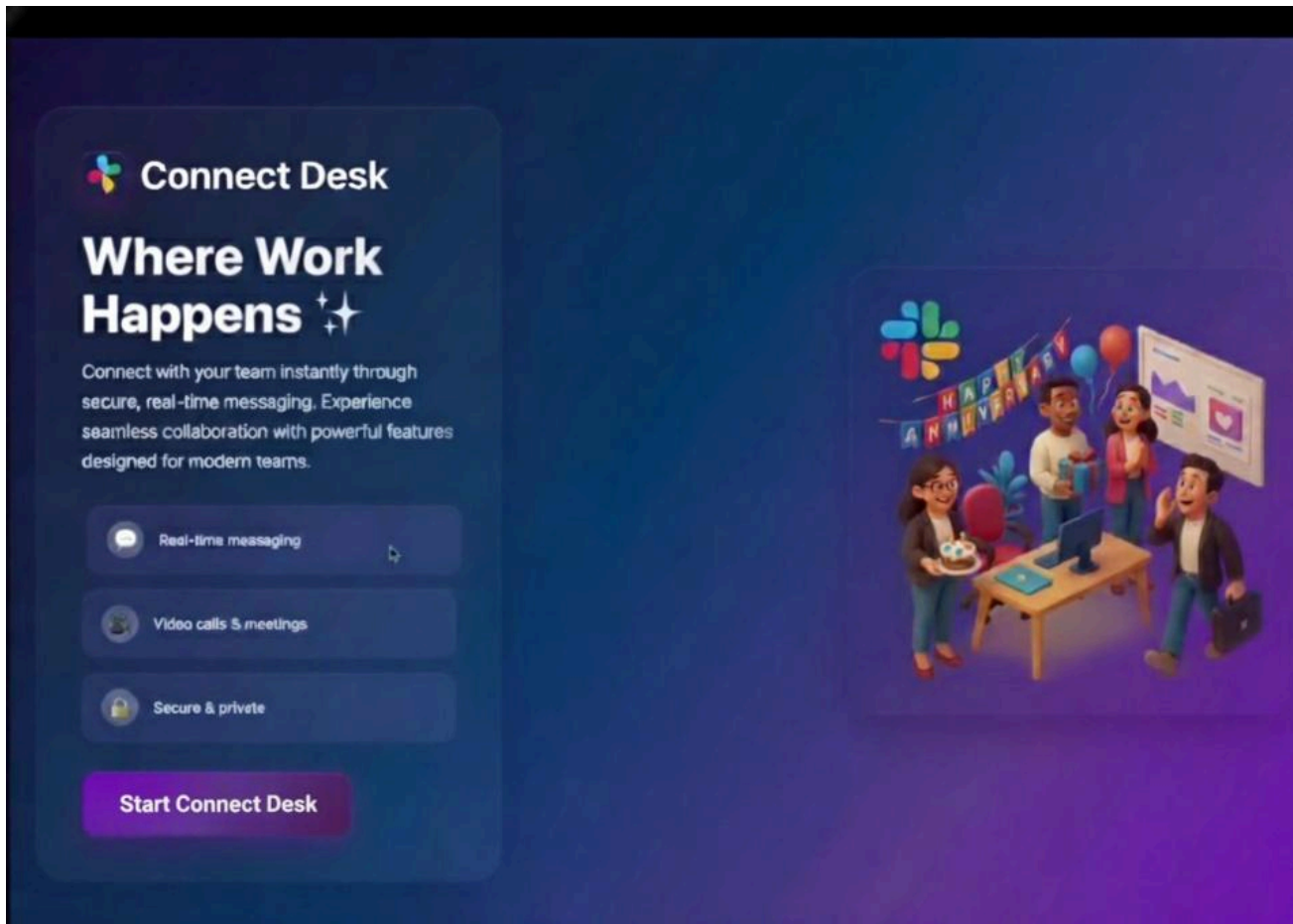


Figure 6.2: Main Application Chat Interface (Description) This screenshot displays the main "Connect Desk" application interface. It shows the full three-column layout

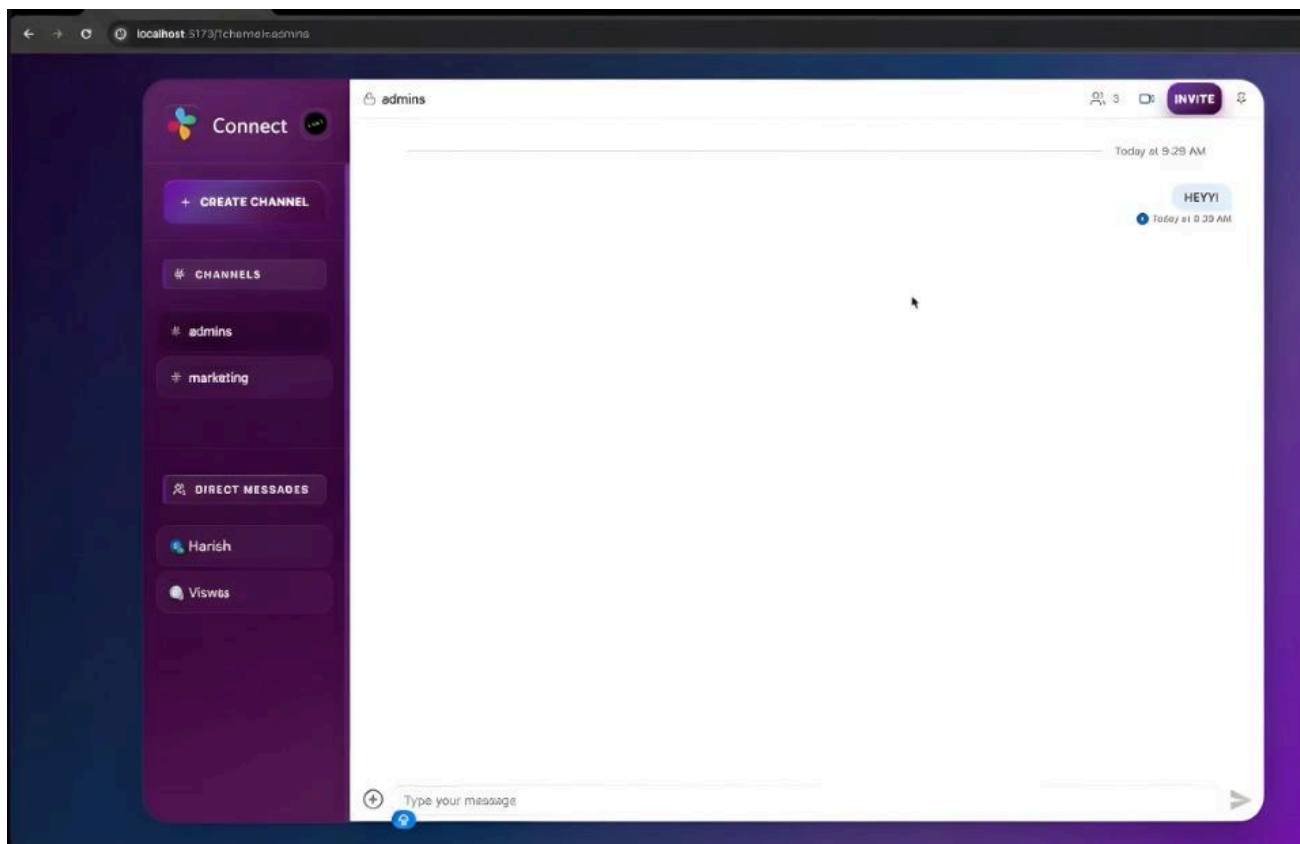


Figure 6.3: File Upload Demonstration (Description) This screenshot shows the chat window after a user has uploaded a file. Instead of text content, the message bubble clearly shows the uploaded image, demonstrating the successful integration with the Cloudinary API and the file-sharing message flow

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

This final chapter wraps up the project, highlights its successes, and considers future possibilities. The conclusion offers a brief overview of the entire project. It confirms that "Connect Desk" was successfully built to meet the project goals and works well as a MERN-stack Slack clone. It summarizes the problems addressed and the solutions applied. The chapter then shifts to the project's limitations and suggests a clear plan for future improvements. It proposes possible new features, such as video calls, threaded messaging, and a mobile app, which could advance the project from a proof-of-concept to a fully functional, ready-for-use platform.

7.1 CONCLUSION:

This project achieved its goal of developing "Connect Desk," a real-time, MERN-stack messaging application inspired by Slack. We met all core objectives: we built a secure, token-based authentication system; designed a scalable MERN-stack architecture; implemented a real-time chat module using Socket.io; and integrated a strong client-side state management system using Redux and Redux-Saga. The project showcases modern full-stack JavaScript development and successfully combines multiple complex technologies, including REST, WebSockets, cloud storage, and authentication, into a single, functional application. The final product is a cost-effective and self-hostable option for team collaboration.

7.2 FUTURE ENHANCEMENT:

While the main features are complete, the "Connect Desk" platform is built for growth. Future work could include:

- Video and Voice Calls: Adding WebRTC to enable one-on-one and group video and voice calls directly in the app.
- Threaded Replies: Changing the Message schema to include a parentMessage reference, so users can reply to specific messages and create threads.
- User Presence & Typing Indicators: Using Socket.io to share presence events (online/offline) and real-time typing indicators to enhance user interaction.

- Mentions and Notifications: Setting up a system to check messages for username mentions and send alerts to those users.
- Mobile Application: Creating a mobile client using React Native. This would share a lot of the business logic (Redux, sagas, API calls) with the web client.

REFERENCES:

- [1] Vaggal, S., & G, R. (2021). "A Comparative Study of MERN and MEAN Stacks for Web Application Development." *International Journal of Engineering and Advanced Technology (IJEAT)*, 10(3), 134-138.
- [2] Sharma, S., & Singh, S. (2020). "Performance Analysis of MERN and LAMP Stacks for Web Application Development." *2020 8th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*, 467-471. DOI: 10.1109/ICRITO48877.2020.9197829.
- [3] Tassabehji, R., & Tassabehji, M. (2022). "Developing Secure Web Applications using MERN Stack." *2022 15th International Conference on Computer Engineering and Applications (ICCEA)*, 120-126.
- [4] P, A., & K, S. (2020). "A Study on Real-Time Web Application Development using Node.js and Socket.io." *International Journal of Computer Applications*, 176(45), 26-30. DOI: 10.5120/ijca2020920405.
- [5] Chandran, A., & Shankar, R. (2019). "Scalable Chat Application using Socket.io and Node.js." *2019 3rd International Conference on Trends in Electronics and Informatics (ICOEI)*, 592-596.
- [6] Hasan, M., & Islam, M. (2018). "Performance Evaluation of MongoDB and MySQL for Web Applications." *International Journal of Computer Science and Information Security (IJCSIS)*, 16(10), 98-103.
- [7] Atre, S. (2021). "A Comprehensive Review on Node.js as a Server-Side Technology." *International Journal of Research in Engineering, Science and Management (IJRESM)*, 4(5), 185-188.
- [8] Al-Janabi, S., & Al-Shara, N. (2019). "Securing RESTful APIs using JSON Web Token (JWT) in a Stateless Environment." *2019 International Conference on New Trends in Computing and Information Technology (NTCIT)*, 258-263.
- [9] Singh, P., & Sharma, V. (2022). "A Review on State Management Techniques in React Applications: Redux vs. Context API." *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, 11(7), 45-50.
- [10] Patel, H., & Patel, V. (2020). "An Analysis of Security Mechanisms for REST API Authentication and Authorization." *International Journal of Computer Science and Network Security (IJCSNS)*, 20(12), 127-133.
- [11] Kumar, A., & Garg, M. (2019). "A Study on Single Page Application (SPA) Development using React.js." *International Journal of Scientific Research in Computer Science, Engineering and Information Technology (IJSRCSEIT)*, 5(2), 762-768.

- [12] Ahmed, A., & M, S. (2021). "Building a Scalable and Secure E-Commerce Platform using MERN Stack." 2021 International Conference on Advances in Computing, Communication, and Control (ICAC3), 1-6.
- [13] O'Brien, L. (2018). "Handling Asynchronous Operations in Redux Applications with Redux-Saga." Journal of Web Development and E-Business, 4(1), 22-34.
- [14] Wang, F., & Li, X. (2019). "Design and Implementation of a Real-Time Web Communication System based on WebSocket." 2019 IEEE 4th International Conference on Big Data Analytics (ICBDA), 320-324.
- [15] Gupta, S., & Dave, M. (2021). "A Framework for Cloud-Based Media Storage and Delivery for Modern Web Applications." Journal of Cloud Computing: Advances, Systems and Applications, 10(42).
- [16] Setia, A., & Sharma, S. (2022). "An Analytical Study of JavaScript Frameworks for Full-Stack Development." 2022 International Conference on Applied and Theoretical Computing and Communication Technology (iCATecT), 1-5.